

NCTU-EE IC LAB – Fall 2026

Lab01 Exercise

Design: Out-of-Order Instruction Scheduling Solver (OISS)

Data Preparation

1. Extract files from TA's directory:

```
% tar -xvf ~iclabTA01/Lab01_2026_F.tar
```

Design Description and Examples

In today's high-performance computing era, processor performance depends not only on clock frequency but also on how efficiently independent instructions are scheduled. Since instructions may have different execution latencies and data dependencies, issuing every instruction strictly in program order can introduce unnecessary stalls and increase the total execution time.

In this lab, you will design an Out-of-Order Instruction Scheduling Solver (OISS). The solver receives eight instructions and their execution latencies, analyzes register dependencies, and determines a legal issue order with the minimum execution cycle count. Independent instructions may overlap in execution, while dependent instructions must preserve the required relationship.

Your design must output the minimum cycle count and one corresponding instruction order. If multiple optimal orders exist, any legal order achieving the same minimum cycle count will be accepted

Description of Lab

➤ Input:

You will receive two input signals:

Inst_seq_I (96 bits):

contains eight 12-bit instructions. Instruction 0 is stored in the least significant 12 bits, and instruction 7 is stored in the most significant 12 bits.

Inst_latency_I (48 bits):

input contains the execution latency of the eight supported opcodes. Each opcode latency is represented by 6 bits, and the bit range is assigned according to the opcode order.

➤ Instruction Format

Each instruction is 12 bits and is divided into four 3-bit fields:

[11:9] opcode		[8:6] rs		[5:3] rt		[2:0] rd
---------------	--	----------	--	----------	--	----------

Detailed instructions are shown below:

Function Name	Meaning	Instruction Binary Encode
ADD	$rd = rs + rt$	000-rs-rt-rd
SUB	$rd = rs - rt$	001-rs-rt-rd
MUL	$rd = rs * rt$	010-rs-rt-rd
DIV	$rd = rs / rt$	011-rs-rt-rd
Load	$rd = DM[\{2'b00, rs, rt\}]$	100-rs-rt-rd
Store	$DM[\{2'b00, rs, rt\}] = rd$	101-rs-rt-rd
Branch on equal	If $(rs == rt)$ $pc = pc + 1 + rd$ else $pc = pc + 1$	110-rs-rt-rd
Jump	Next instruction address $pc = pc + \{rs, rt, rd\}$	111-rs-rt-rd

In load, store, branch, and jump instructions, the registers marked in red (rs, rt, rd) represent their indices rather than the values stored within them; consequently, there are no dependency issues.

BRANCH and JUMP update only the architectural PC variable in PATTERN.v. They do not redirect which of the eight instructions will be executed.

➤ Execution Latency

For every test case, the latency of each instruction is independently generated within the following legal range:

Instruction	Execution Latency (cycles)
ADD	1 - 5
SUB	1 - 5
MUL	20 - 40
DIV	30 - 50
LOAD	6 - 10
STORE	6 - 10

Instruction	Execution Latency (cycles)
BRANCH	2 - 4
JUMP	1

PATTERN.v guarantees that every latency value is legal for its corresponding opcode. Within the same test pattern, all instructions with the same opcode share the same execution latency. The latency assigned to each opcode may vary between different test patterns. Since identical instruction encodings may appear at different positions in the input sequence, the output order must be represented using the original instruction indices.

➤ Input Packing

Original Index	Instruction Bit Range	Opcode	Latency Bit Range
Instruction 7	Inst_seq_I[95:84]	ADD	Inst_latency_I[5:0]
Instruction 6	Inst_seq_I[83:72]	SUB	Inst_latency_I[11:6]
Instruction 5	Inst_seq_I[71:60]	MUL	Inst_latency_I[17:12]
Instruction 4	Inst_seq_I[59:48]	DIV	Inst_latency_I[23:18]
Instruction 3	Inst_seq_I[47:36]	LOAD	Inst_latency_I[29:24]
Instruction 2	Inst_seq_I[35:24]	STORE	Inst_latency_I[35:30]
Instruction 1	Inst_seq_I[23:12]	BRANCH	Inst_latency_I[41:36]
Instruction 0	Inst_seq_I[11:0]	JUMP	Inst_latency_I[47:42]

➤ Output

The DUT must output one optimal issue order and its minimum execution cycle count.

Field	Bit Range	Meaning
Issue position 0	Inst_order_O[2:0]	Original instruction index (0 - 7)
Issue position 1	Inst_order_O[5:3]	Original instruction index (0 - 7)
Issue position 2	Inst_order_O[8:6]	Original instruction index (0 - 7)
Issue position 3	Inst_order_O[11:9]	Original instruction index (0 - 7)
Issue position 4	Inst_order_O[14:12]	Original instruction index (0 - 7)
Issue position 5	Inst_order_O[17:15]	Original instruction index (0 - 7)
Issue position 6	Inst_order_O[20:18]	Original instruction index (0 - 7)
Issue position 7	Inst_order_O[23:21]	Original instruction index (0 - 7)

The least significant 3-bit field represents the first instruction to be issued. Every original instruction index from 0 to 7 must appear exactly once.

Ex_cycle[8:0] = the minimum completion cycle count achieved by Inst_order_O

➤ Register Dependency

For any pair of instructions I_i and I_j in the original input sequence, where $i < j$, the original order $I_i \rightarrow I_j$ must be preserved if at least one of the following dependencies exists:

Type	Name	Detection Condition	Meaning
RAW	Read After Write	WriteSet(I_i) intersects ReadSet(I_j)	I_j needs a value produced by I_i
WAR	Write After Read	ReadSet(I_i) intersects WriteSet(I_j)	I_i must read the old value before I_j overwrites it
WAW	Write After Write	WriteSet(I_i) intersects WriteSet(I_j)	The final write order must remain unchanged

For every dependency $I_i \rightarrow I_j$: start_cycle[I_j] $\geq$ finish_cycle[I_i]

➤ ReadSet and WriteSet

Instruction	ReadSet	WriteSet
ADD / SUB / MUL / DIV	{rs, rt}	{rd}
LOAD	None	{rd}
STORE	{rd}	None
BRANCH	{rs, rt}	None
JUMP	None	None

For STORE, rd is the source data register, not a destination register. For BRANCH, the lowest 3-bit field is an immediate offset. For JUMP, the lower 9 bits are an immediate value; therefore, JUMP does not read or write any register.

➤ Memory Dependency Assumption

No memory dependency occurs in this lab. PATTERN.v guarantees that reordering LOAD and STORE instructions does not change any load result or final memory value. The generated test cases ensure that memory operations access non-conflicting addresses.

Register dependencies used to calculate a memory address or store data are still considered. For example, if an ADD produces the address register used by a later LOAD, a RAW dependency exists between the two instructions.

➤ Dependency-Graph Constraint

For every test pattern, one of the following structures is guaranteed:

- 1.No RAW, WAR, or WAW dependency exists among the eight instructions.
- 2.Exactly one dependency chain of length 2 to 8 exists, while all remaining instructions are completely independent.
- 3.Exactly two vertex-disjoint dependency chains exist, and the two chains together contain all eight instructions.

Instructions in the same dependency chain must preserve their dependency order. No dependency edge exists between different chains or between a chain and an independent instruction. Multi-cycle instructions may appear at any position in a dependency chain and may have both incoming and outgoing dependencies.

➤ Scheduling Model

Single issue:

at most one instruction can begin execution in each cycle.

Overlapped execution:

previously issued independent instructions may execute concurrently.

No structural hazard:

functional units are assumed to be available whenever an instruction is ready. Two instructions of the same type may overlap in execution.

Strict output order: Inst_order_O specifies the exact issue order. If the next instruction in the output order is not ready, the issue stage must stall and cannot skip to a later instruction.

➤ Cycle Calculation

The first instruction may start at cycle 0. If an instruction starts at cycle S and has latency L, it executes during [S, S + L) and completes at cycle S + L.

$$\text{finish_cycle}[\text{current}] = \text{start_cycle}[\text{current}] + \text{latency}[\text{current}]$$



$$\text{start_cycle}[\text{order}[0]] = 0$$



$$\text{start_cycle}[\text{order}[k]] = \max(\text{start_cycle}[\text{order}[k-1]] + 1, \text{finish_cycle of every dependency predecessor})$$



$$\text{Ex_cycle} = \text{maximum finish_cycle among all eight instructions}$$

A dependent instruction may start exactly at the finish cycle of its predecessor. Because the longest possible serial chain is eight 50-cycle instructions, a 9-bit

Ex_cycle is sufficient to represent the maximum value of 400 cycles.

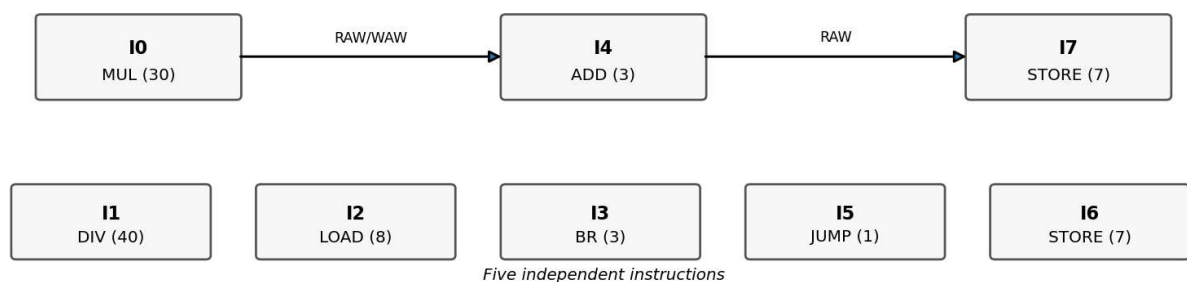
Multiple optimal answers are allowed. No index-based tie-breaking rule is required.

➤ Ideal Example

This example contains one dependency chain and five independent instructions. The initial register and memory values are omitted because they do not affect the dependency graph.

Index	Instruction	Latency	Dependency Notes
I0	MUL R1, R2, R3	30	Writes R1; reads R2 and R3
I1	DIV R5, R6, R7	40	Independent
I2	LOAD R0, DM[8'h12]	8	Independent
I3	BRANCH R6, R7, +2	3	Independent
I4	ADD R1, R1, R4	3	RAW/WAW from I0
I5	JUMP +1	1	Independent
I6	STORE DM[8'h09], R4	7	Independent
I7	STORE DM[8'h2A], R1	7	RAW from I4

➤ Example Dependency Graph

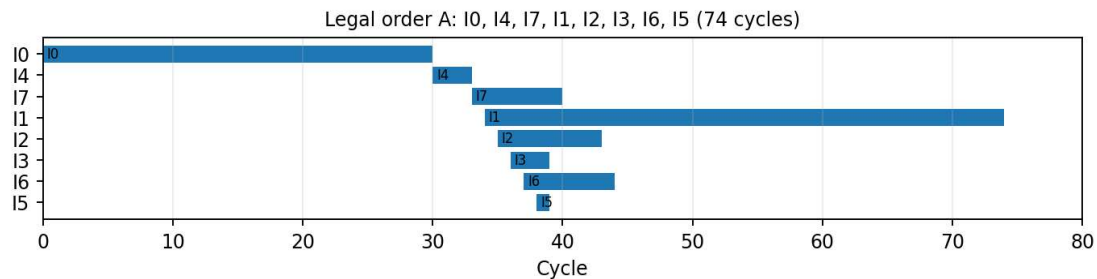


➤ Legal but Non-Optimal Order

Order A = I0, I4, I7, I1, I2, I3, I6, I5 Ex_cycle = 74

Order A preserves the chain order I0 -> I4 -> I7. However, I4 is selected immediately after I0 and cannot begin until I0 finishes at cycle 30. Because the output

order is strict, the independent DIV instruction I1 cannot be issued during this waiting interval. I1 therefore starts at cycle 34 and becomes the last completing instruction at cycle 74.

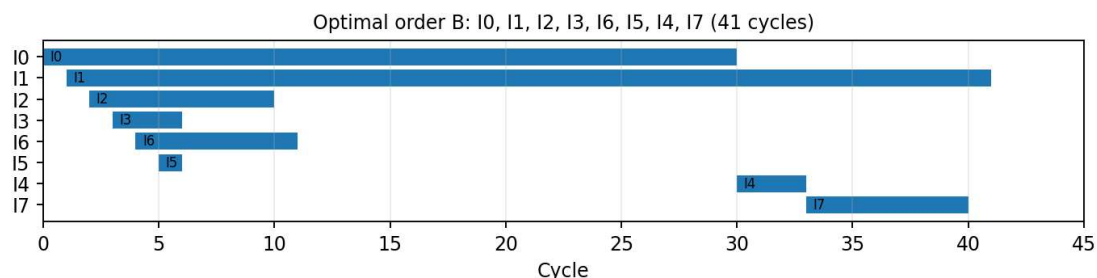


Instruction	Start Cycle	Finish Cycle
I0	0	30
I4	30	33
I7	33	40
I1	34	74
I2	35	43
I3	36	39
I6	37	44
I5	38	39

➤ Optimal Order

Order B = I0, I1, I2, I3, I6, I5, I4, I7 Ex_cycle = 41

Order B issues the independent instructions while I0 is executing. I4 starts as soon as I0 completes, and I7 starts as soon as I4 completes. The result cannot be less than 41 cycles: I0 and I1 cannot both start at cycle 0. Starting either one first forces the other long path to complete no earlier than cycle 41.



➤ Output Encoding for the Example

Output Field	Value
Inst_order_O[2:0]	3'd0
Inst_order_O[5:3]	3'd1
Inst_order_O[8:6]	3'd2
Inst_order_O[11:9]	3'd3
Inst_order_O[14:12]	3'd6
Inst_order_O[17:15]	3'd5
Inst_order_O[20:18]	3'd4
Inst_order_O[23:21]	3'd7
Ex_cycle	9'd41

Inputs

Signal	I/O	Bit Width	Description
Inst_seq_I	Input	96	Eight 12-bit instructions; instruction 0 is stored in [11:0].
Inst_latency_I	Input	48	Eight 6-bit opcode latencies.
Inst_order_O	Output	24	One optimal issue order; eight original indices, 3 bits each.
Ex_cycle	Output	9	Minimum completion cycle count achieved by Inst_order_O.

1. Inst_seq_I is a 96-bit input containing eight 12-bit instructions.
2. Inst_latency_I is a 48-bit input containing the latency of each original instruction.
3. The input sequence may contain RAW, WAR, and WAW dependencies.
4. PATTERN.v guarantees that no memory dependency occurs and that every latency value is within the legal opcode range.

Output

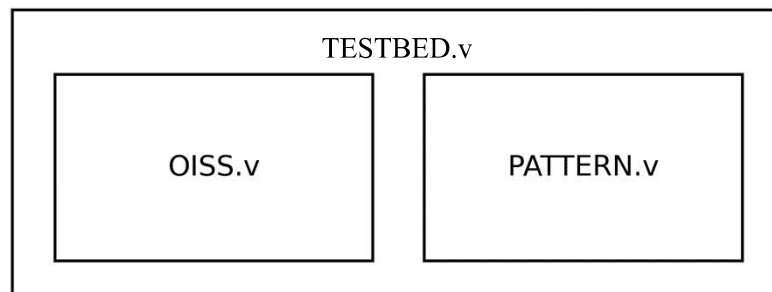
Inst_order_O is a 24-bit concatenation of eight original instruction indices. The least significant field is issued first. Ex_cycle is the minimum completion cycle count achieved by the output order.

Specifications

1. Top module name: OISS (File name: OISS.v).
2. After synthesis, check OISS.area and OISS.timing in the Report folder. The area report is valid only when the final timing slack is MET.
3. The synthesis result cannot contain any latch.
4. Hardcoding answers or using a lookup table based on the test case index is prohibited.
5. Synopsys IP and comments beginning with // synopsys are prohibited in this lab.
6. You can adjust your clock period by yourself, but the maximum period is 100 ns. The precision of clock is 0.1, for example, 79.5 is allowed but 79.55 is not.

7. The total cell area should not larger than 1500,000 μm^2 .
8. Only English comments are allowed.
9. The DUT must be a purely combinational design.
10. Hidden cases will include conditions beyond the provided open cases.
11. Don't use any wire/reg/submodule/parameter name called ***error***, ***Congratulations***, ***latch*** or ***FAIL*** otherwise you will fail the lab. Note: * means any char in front of or behind the word. e.g: error_note is forbidden.

Block Diagram



Grading Policy

The performance is determined by the area of your design. The less area your design has, the higher grade you get. Try to reach better performance by thinking your architecture before coding.

Function Validity: 70%

Performance: cycle_time*area 30%

If you fail Lab01 at first demo, and pass at second demo, you will only get 50 point. Get no score if you fail both first and second demo. Note that you will get 0 score if you are found plagiarism at your code.

Note

1. Tar all your design by run the command **Lab01/09_SUBMIT/00_tar**
2. Submit your design through **Lab01/09_SUBMIT/01_submit**
 - a. 1st_demo deadline: **2026/09/21(Mon.) 12:00:00**
 - b. 2nd_demo deadline: **2026/09/23(Wed.) 12:00:00**
3. If your file **violates the naming rule**, you will **lose 5 points**.

Be careful about all details!

Template folders and reference commands:

In demo, the reference commands is:

1. 01_RTL (RTL simulation):

./01_run_vcs_rtl

2. 02_SYN/ (Synthesis):

./01_run_dc_shell

(Check **latch** by searching the keyword “**Latch**” in 02_SYN/**syn.log**)

(Check the design’s timing in /Report/ SMC.timing)

(Check the design’s area in /Report/ SMC.area)

3. 03_GATE/ (Gate-level simulation):

./01_run_vcs_gate

4. 09_SUBMIT/ (submit your files):

./00_tar

./01_submit

./02_check

You can key in **./09_clean_up** to clear all log files and dump files in each folder

Example Waveform

Input and output signal:

Ex_cycle[8:0]	7	39	23	1a	22	1c
Inst_order_0[23:0]	fa_c688	f3_2629	9c_e750	73_70a9	3a_077a	d3_ba11

Hint

Hint1: Pattern provided by TA will cover only some simple cases, you can try to write your own input / output file by yourself. Here is the format how TA will read in

PATTERN, take care to use hexadecimal numbers to generate the pattern:

input.txt format

1. [PATTERN_NUM]

repeat (PATTERN_NUM)

```
1. [Inst0] [Inst1] [Inst2] [Inst3]
   [Inst4] [Inst5] [Inst6] [Inst7]

2. [ADD_latency] [SUB_latency]
   [MUL_latency] [DIV_latency]
   [LOAD_latency] [STORE_latency]
   [BRANCH_latency] [JUMP_latency]
```

```
100 ← pattern number
34b dfb 04c 3c7 ad6 8b5 c69 897
2 5 29 30 8 8 3 1 ← inst
3b9 df4 dc3 40e e91 eea 3fc d3e
4 2 29 38 6 6 2 1
```

output.txt format

repeat (PATTERN_NUM)

1. [minimum_cycle]

```
17
32 ← minimum cycle
57
58
77
57
49
```

Hint2: If you want to change your cycle time, make sure you change the following parts in your directory, and run the 03_GATE to check

